

GOVERNMENT CHALLENGE SUBMISSION

SENTINEL VISION

Unified CCTV Intelligence & Real-Time Smart Policing
Platform

High-Level Design / Architecture Document

Version 1.0 · September 2026

Repository: github.com/RuudyLinux/SENTINEL-AI (branch `demo-readiness-phase2` , PR #2)

Every capability in this document is checked against the code. Items are labelled **Current implementation**, **Demonstrated capability** or **Planned / future scale**.

Contents

1. Executive Summary	13. GIS / Camera Map
2. Problem Context	14. Self-Heal Architecture
3. Solution Overview	15. Security Architecture
4. System Architecture	16. Database Architecture
5. Component Architecture	17. API / WebSocket Architecture
6. Camera / VMS Integration	18. Deployment Architecture
7. Video Processing Pipeline	19. Current Limitations
8. AI Analytics	20. Scalability Roadmap
9. ANPR Pipeline	21. Technology Stack
10. Rules & Event Engine	22. Testing & Verification
11. Alert & Incident Management	23. Conclusion
12. Evidence Management	

Labels used throughout: **CURRENT** implemented in the repository · **DEMONSTRATED** shown working on real or recorded camera footage · **FUTURE** planned, not implemented · **LIMITATION** known issue.

1. Executive Summary

SENTINEL VISION is a working CCTV intelligence platform. It registers cameras from different sources in one catalogue, runs AI analytics on their video, turns detections into rule-based alerts and incidents, captures tamper-evident evidence, and gives control-room operators one web dashboard for live monitoring, mapping and investigation.

The system runs today as a FastAPI backend with a per-camera async pipeline, Ultralytics YOLO11s detection on a shared GPU model, ByteTrack tracking, a plate detector with EasyOCR for number plates, and a Next.js 16 dashboard. It has been exercised against a real 30-camera Sentinel Grid catalogue as well as local video, webcam and RTSP sources.

This document describes the architecture as it exists, the measured behaviour, and the known limitations. Scale-out items (multi-node workers, edge AI, message bus) are described in Section 20 and are **not** implemented.

2. Problem Context

- **Fragmented CCTV.** Cameras sit behind different vendors, recorders and VMS products, each with its own interface.
- **Manual monitoring.** Operators watch many feeds at once; attention does not scale with camera count.
- **Late identification.** Events such as a watch-listed vehicle or an intrusion into a restricted area are often found only after the fact.
- **Slow investigation.** Reconstructing where a vehicle went means scrubbing footage camera by camera and collecting evidence by hand.
- **Unreliable connectivity.** Streams drop, credentials expire and bandwidth varies; a monitoring system has to notice and recover on its own.

3. Solution Overview

Capability	What SENTINEL VISION does	Status
Unified camera registry	One catalogue for webcam, video file, RTSP, Sentinel Grid and mock VMS sources, with catalogue sync, status and health per camera.	CURRENT
AI video analytics	Person and vehicle detection (YOLO11s), multi-object tracking (ByteTrack), plate localisation and OCR (EasyOCR), switchable per camera.	DEMONSTRATED
Rules and events	Restricted zones, watchlists, loitering and a risk score produce alerts; CRITICAL alerts open or join incidents.	CURRENT
Evidence	Snapshots, event clips and operator REC recordings, hashed at capture, verified on access, packaged per incident.	DEMONSTRATED
GIS map	Leaflet/OpenStreetMap camera map and vehicle journey maps; 19 of 30 grid cameras placed.	CURRENT
Self-Heal	Camera health states, reconnect with backoff, grid-wide breaker, database retry, recovery events visible to operators.	CURRENT
Security	JWT login, five roles with per-route RBAC, short-lived stream/evidence tokens, hash-chained audit log.	CURRENT

4. System Architecture

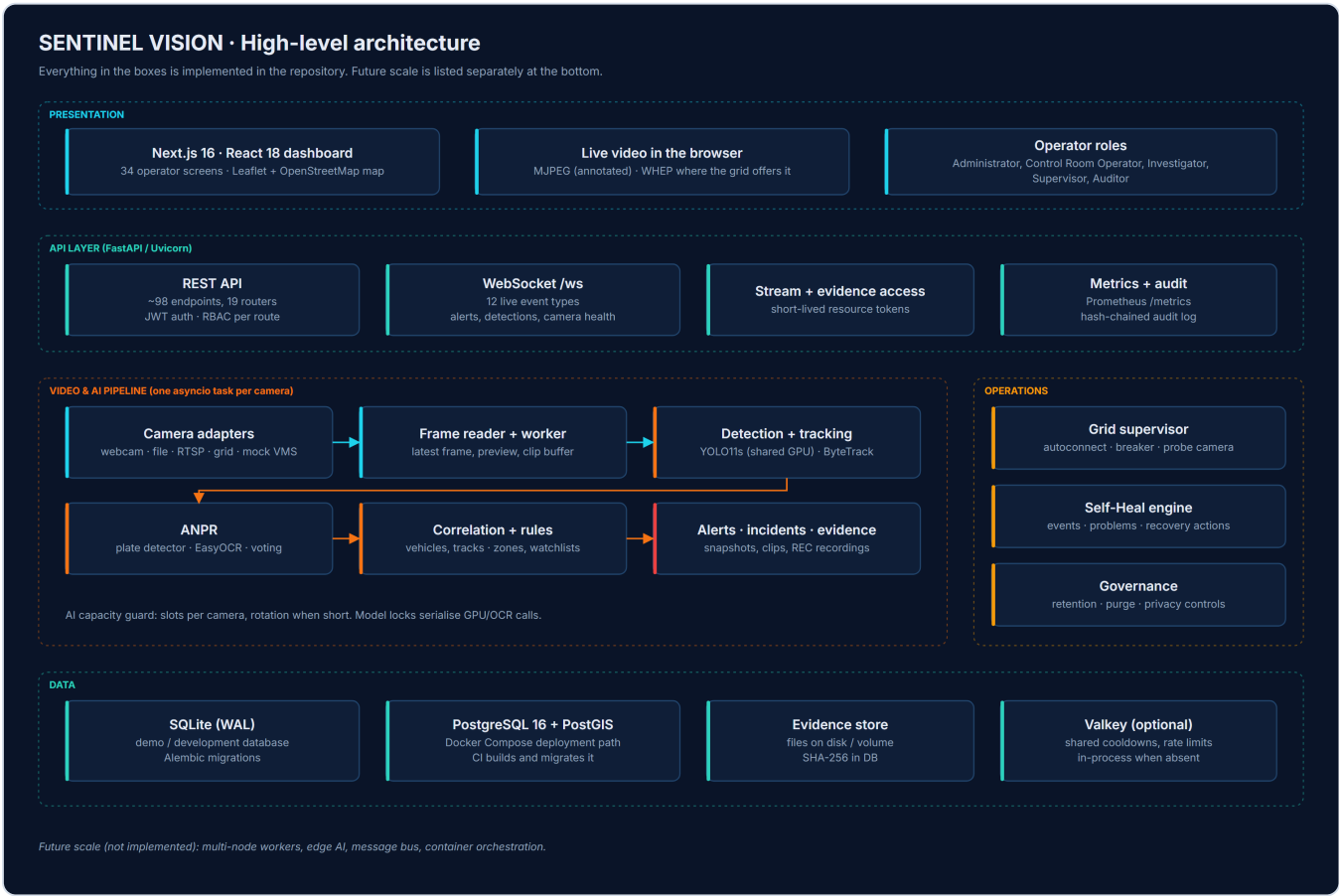


Figure 1. High-level architecture. Every box is implemented; future items are listed only at the bottom.

The system has four tiers. The **presentation** tier is a Next.js 16 / React 18 web application with 34 operator screens. The **API** tier is FastAPI: about 98 REST endpoints across 19 routers, a WebSocket at `/ws`, and token-protected stream and evidence routes. The **pipeline** tier runs one async task per connected camera inside the backend process. The **data** tier is a relational database (SQLite in the demo, PostgreSQL + PostGIS in the Docker Compose deployment), an evidence file store, and an optional Valkey (Redis-protocol) store for shared runtime state.

5. Component Architecture

Component	Responsibility	Code
Camera adapters	Open and read each source type; RTSP transport and buffer options; grid URL built in memory only.	<code>pipeline/adapters.py</code> , <code>source.py</code>
Grid supervisor	Catalogue discovery, autoconnect, staggered connects, escalating breaker with a probe camera.	<code>pipeline/supervisor.py</code> , <code>sentinel_grid.py</code>
Frame reader	Dedicated thread per camera keeps only the latest frame; grabs without decoding when nothing needs pixels.	<code>pipeline/frame_reader.py</code>
Camera worker	Per-camera loop: health state, preview, clip ring buffer, AI, persistence, heartbeat.	<code>pipeline/worker.py</code>
AI capacity guard	Grants AI slots per camera; rotates slots when more cameras want AI than the hardware can serve.	<code>pipeline/ai_capacity.py</code>
Detector / tracker	One shared YOLO11s model, one ByteTrack tracker per camera.	<code>pipeline/detector.py</code>
ANPR	Plate localisation, preprocessing variants, EasyOCR, per-track vote.	<code>plate_detector.py</code> , <code>anpr.py</code> , <code>plate_tracker.py</code>
Correlation	Vehicles, plate sightings, tracks, cross-camera journeys, appearance similarity.	<code>pipeline/correlate.py</code> , <code>appearance.py</code>
Rules engine	Zones, watchlists, loitering, cooldowns, risk score, alert and incident creation.	<code>pipeline/rules_engine.py</code> , <code>risk.py</code>
Clips / recorder	Event clips around alerts; operator REC button records annotated MP4 evidence.	<code>pipeline/clips.py</code> , <code>recorder.py</code>
Self-Heal	Failure classification, events, open problems, recovery reporting.	<code>app/self_heal/</code>
Security	Password hashing, JWT, resource tokens, RBAC dependencies, audit chain.	<code>security.py</code> , <code>audit.py</code>

6. Camera / VMS Integration

CURRENT Cameras live in one registry table with code, name, location, coordinates, source type and AI switches. Adapters hide how each source is read, so the rest of the pipeline sees the same frames whatever the camera is.

Source	How it is integrated	Status
Webcam	Local device index through OpenCV.	CURRENT
Video file	Uploaded through the dashboard, validated, looped as a continuous feed.	CURRENT
RTSP	OpenCV/FFmpeg with configurable TCP or UDP transport and receive buffer.	CURRENT
Sentinel Grid	Cookie login to the grid, catalogue sync of 30 cameras, RTSP over UDP, WHEP live video where offered. Credentials only in <code>.env</code> , never logged or returned.	DEMONSTRATED
Mock VMS	Synthetic feed that proves the generic VMS adapter interface.	CURRENT
ONVIF / other VMS	Interface stub only; a new VMS is added as another adapter behind the same interface.	FUTURE

Grid status (known, not fully resolved). Grid authentication is only partly working. In recent runs about 29 cameras sat in a degraded (reconnecting) state while a few connected, with roughly 3.4 Mbit/s of total video received. At other times the grid rejected the account outright (HTTP 401/403). In one earlier controlled test over UDP, 27 of 30 streams delivered about 51 Mbit/s in total; over TCP the account was capped near 5 Mbit/s. The platform does not claim 30 simultaneous grid streams.

7. Video Processing Pipeline

Each connected camera gets one async task and one reader thread. The reader keeps only the newest frame, so a slow consumer never builds a backlog. When nobody watches the camera and AI is off, it grabs packets without decoding them, which measured 5.9 ms per 1080p HEVC frame against 12.9 ms for a full decode.

1. Read the latest frame and its stream-relative timestamp.
2. Update health: CONNECTED (frames, AI off), PROCESSING (frames and AI), DEGRADED, RECONNECTING, AUTH_ERROR, DISCONNECTED.
3. Push a frame to the clip ring buffer and refresh the annotated preview for viewers.
4. If the camera holds an AI slot, run detection and tracking, then ANPR, persistence and rules.
5. Commit a heartbeat (last frame time, status) at most every 2 seconds.

8. AI Analytics

CURRENT Detection uses Ultralytics **YOLO11s** (COCO classes: person, car, motorbike, bus, truck) at 960 px. YOLOv8s was the earlier baseline and was replaced after benchmarking. One model instance is shared across cameras on the GPU under a lock; each camera keeps its own **ByteTrack** tracker, so track IDs never mix between cameras. Measured GPU inference was about 24 ms per frame on the development laptop (RTX 3050 Ti).

Selective analytics. Person, vehicle and ANPR analytics are switched on per camera. When more cameras want AI than the hardware can serve, the capacity guard rotates AI slots between them (60 s turns) instead of starving some cameras permanently.

Measured detection quality, with its caveats. On the project's internal benchmark of grid frames, YOLO11s at 960 px scored vehicle precision 0.860 / recall 0.508 and person precision 0.959 / recall 0.538 (docs/AI_ACCURACY.md). The reference labels were built from agreeing model runs and reviewed by an AI assistant, not trained annotators, so these figures are an engineering comparison between configurations, not a certified accuracy.

9. ANPR Pipeline

1. A vehicle detection with a track ID is cropped from the frame.
2. A fine-tuned YOLO plate detector localises the plate; perspective correction applies to skewed plates.
3. Preprocessing builds image variants; **EasyOCR** reads each and the best-agreeing read wins.
4. A read must pass a gate (Indian plate format, state codes, confidence) to count.
5. Reads are **voted** across frames of the same track. One lucky frame is stored as uncorroborated and goes to review; a CRITICAL watchlist alert needs corroboration across frames.
6. The sighting links to a vehicle record, which builds the cross-camera journey.

OCR runs before any database write for the detection, so slow OCR never holds the database write lock.

10. Rules & Event Engine

Rule	Behaviour
Restricted zone	A confident detection inside an active zone, within its schedule, seen for at least two frames of the same track.
Watchlist plate	Fires only for an active watchlist entry (the entry is the authority, not a cached flag). Severity is capped at HIGH until the plate read is confident and corroborated.
Loitering	Dwell time in a zone above the zone's threshold, where an active loitering rule targets it.
Risk score	Combines zone severity, watchlist priority, plate confidence and vehicle history; it can raise severity, never lower a rule's CRITICAL.

Cooldowns per camera, rule and track stop one object from raising the same alert every frame. With Valkey configured, cooldowns are shared across backend processes.

11. Alert & Incident Management

CURRENT An alert stores its reasons, severity, risk factors, detection, vehicle and snapshot, and is pushed to every dashboard over the WebSocket. Operators acknowledge, escalate or dismiss alerts, and feedback is recorded. A CRITICAL alert opens an incident or joins an open one for the same vehicle (or camera) within the correlation window, so one vehicle crossing four cameras becomes one incident whose title and priority grow with it. Incidents carry status, notes and a summary endpoint.

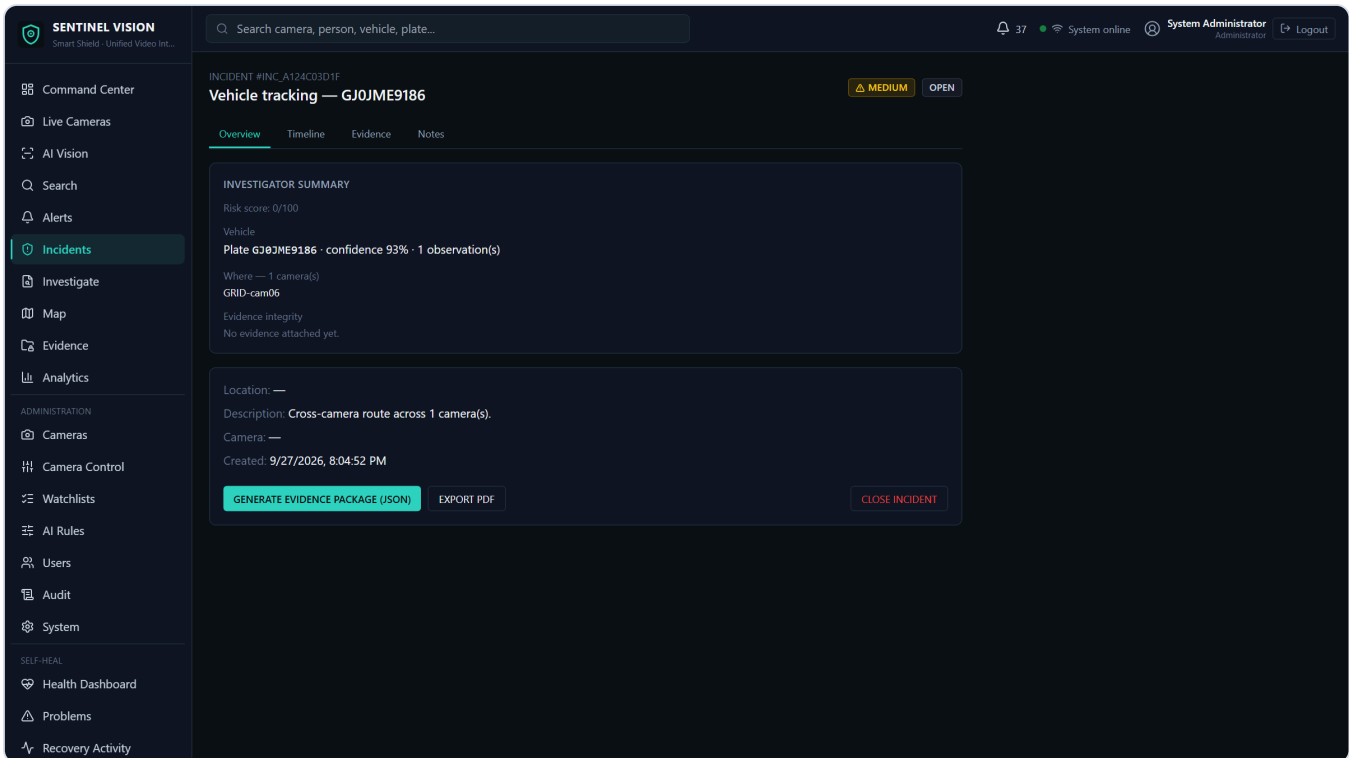


Figure 2. Incident detail from the running system.

12. Evidence Management

- **Snapshot** for every alert, from the same frame.
- **Event clip** assembled from the pre-event ring buffer and post-event frames.
- **REC recording**: the operator starts and stops an annotated MP4 recording on a live camera; it is saved as evidence.
- **Integrity**: SHA-256 computed at capture, re-verified on access (VERIFIED badge), model and rule versions stamped on the record.
- **Access**: evidence files open only with a short-lived signed token (300 s), and every access is audited.
- **Evidence package** per incident: summary, camera timeline, vehicle details, evidence list, notes and audit trail, as JSON or PDF, with optional plate redaction for wider distribution.
- **Governance**: retention and purge controls (`routers/governance.py` , `docs/PRIVACY_GOVERNANCE.md`).

13. GIS / Camera Map

CURRENT The map uses Leaflet with OpenStreetMap tiles. The Sentinel Grid catalogue supplies no coordinates, so camera positions were looked up once in OpenStreetMap (Nominatim and Overpass) from each camera's place name and stored in `pipeline/grid_locations.json`, with the precision of each point.

- **19 of 30** grid cameras have usable positions: 10 at a landmark or junction, 5 at neighbourhood level, 4 at town level only.
- **11 cameras are intentionally not mapped** because no reliable match was found. They are not guessed; the map states how many are not shown.
- A position is applied only if the camera's catalogue name still matches, and never overwrites a position an operator set.
- The map fits all mapped cameras; cameras sharing one position are combined into a single marker with a count (for example the three Bilimora cameras).
- Status colours with a legend: online (green), degraded/reconnecting (amber), offline (grey).
- The vehicle journey map shows sightings on the cameras' positions; GRID-cam06 was verified at Timbavadi, Junagadh.
- Nearby-camera search uses PostGIS `ST_DWithin` on PostgreSQL and a haversine fallback on SQLite.

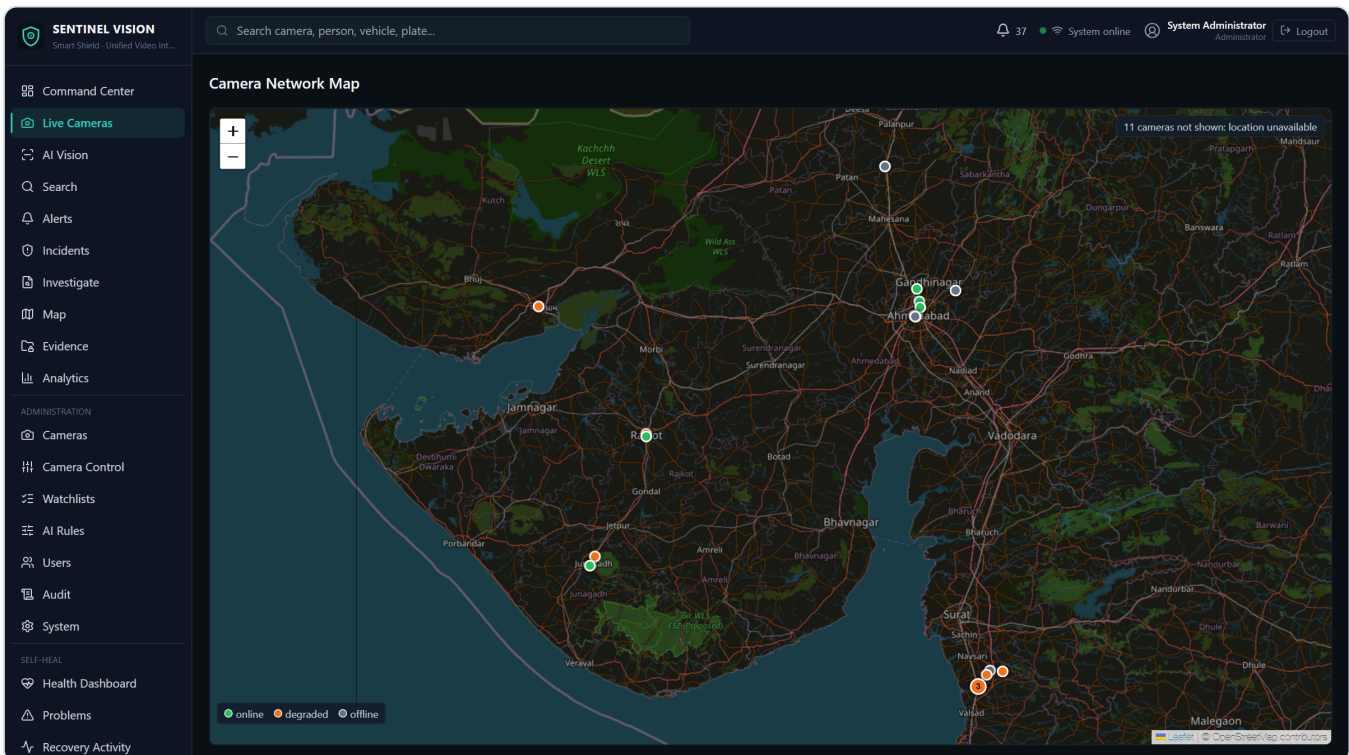


Figure 3. Camera network map: 19 cameras placed across Gujarat, grouped marker for Bilimora, 11 reported as not shown. Change in PR #2.

14. Self-Heal Architecture

Failure	Detection and recovery
Stream drop / read failure	Camera goes DEGRADED, then RECONNECTING with per-camera backoff; recovers on its own when frames return.
Grid-wide rejection	Breaker pauses all grid connects for 5 minutes, doubling on each repeat up to 60 minutes; after a pause one probe camera must connect before the rest try. This protects the grid account from connect storms.
Database lock	Commit/flush retry with rollback and reapply of the pending write; each lock recovery is recorded as a Self-Heal event.
Worker crash	Last-resort guard logs the traceback, marks the camera offline and frees its AI slot; other cameras are unaffected.
WebSocket drop	The dashboard reconnects on its own with bounded exponential backoff.

Every failure is classified and stored as a Self-Heal event; repeated failures become an open "problem" with its history, shown on the Health Dashboard, Problems, Recovery Activity, Error Logs and Camera Health screens.

15. Security Architecture

- **Authentication:** bcrypt password hashes, JWT bearer tokens, login rate limiting per user and IP.
- **RBAC:** five roles (Administrator, Control Room Operator, Investigator, Supervisor, Auditor) enforced per route on the server; the UI only mirrors it.
- **Resource tokens:** live streams (1 h) and evidence files (5 min) use short-lived signed tokens scoped to one resource, so a URL in a browser history is not a standing credential.
- **Audit log:** every sensitive action is written to an append-only, hash-chained log; `GET /api/audit/verify-chain` detects any edited or deleted row.
- **Secrets:** grid and database credentials come from the environment only; the credentialed RTSP URL is built in memory and never stored, logged or returned. Logs pass through redaction.
- **Threat model** in `docs/THREAT_MODEL.md` ; CI runs CodeQL analysis.

16. Database Architecture

SQLAlchemy models cover cameras, detections, vehicles, plates, tracks, alerts, incidents (with alert links and notes), evidence, zones, rules, watchlists, users/roles, audit logs and Self-Heal events. Alembic manages migrations.

Database	Role	Status
SQLite (WAL)	The demo and development database; used in the demonstration shown with this submission.	CURRENT
PostgreSQL 16 + PostGIS	The Docker Compose deployment path. CI builds the stack, runs Alembic to head, and checks the schema has no drift. PostGIS adds an indexed camera geography column.	CURRENT (not load-tested with a live camera fleet)
Valkey	Optional shared runtime state (cooldowns, rate limits); in-process when absent.	CURRENT

SQLite write contention (known limitation, recently fixed). SQLite allows one writer at a time. With the grid cameras connected, logins were observed taking 18 to 149 seconds. The measured cause was a camera holding the write lock while its commit waited for a free thread on a pool busy with AI work. A fix (commit `795ebc8`) gives database operations their own threads, moves snapshot I/O out of open transactions, stops a blocking flush on the event loop, and throttles error-count writes. It is covered by regression tests. Afterwards, login measured 0.25 s median with 8 cameras; it has **not yet been re-verified with all 30 grid cameras streaming**, because grid access was blocked at the time. SQLite remains a single-writer database; PostgreSQL is the recommended production database for a larger fleet.

17. API / WebSocket Architecture

REST (FastAPI, about 98 endpoints): auth, users, cameras and camera control (bulk connect, start, AI, restart, disconnect), catalogue sync, streams, detections, vehicles, persons, alerts, incidents, evidence, zones, rules, watchlists, review, search, analytics, audit, governance, Self-Heal, system diagnostics and Prometheus metrics. OpenAPI documentation is served at `/docs` .

WebSocket `/ws` (authenticated) pushes 12 event types: `detection.created` , `detection.batch` , `plate.detected` , `plate.updated` , `vehicle.sighting` , `vehicle.route.updated` , `alert.created` , `incident.created` , `camera.status` , `camera.health` , `self_heal.recovery` and `bulk_progress` .

Video to the browser: annotated MJPEG per camera (token-protected); WHEP WebRTC where the grid offers it.

18. Deployment Architecture

Mode	What runs	Status
Single machine (demo)	<code>start.bat</code> : backend (Uvicorn, GPU environment when CUDA works, CPU otherwise) and the built Next.js frontend. SQLite, evidence on local disk.	DEMONSTRATED
Docker Compose	PostgreSQL + PostGIS, Valkey, backend (migrated on start) and frontend containers. Secrets from the environment; the stack refuses to start without them.	CURRENT (CI-built)
Multi-node / cloud	Separate AI worker nodes, orchestration, managed database.	FUTURE

19. Current Limitations

Limitation	Detail
Grid streaming	Grid authentication partly working; about 29 cameras degraded and about 3.4 Mbit/s observed in recent runs. Not all 30 stream at once.
SQLite concurrency	Single writer. Login contention fixed and tested, but not re-verified with 30 live grid streams.
Map coverage	11 of 30 grid cameras have no reliable location and are not mapped; 4 are town-level only.
Single process	All camera workers run in one backend process on one machine; AI throughput is bounded by that machine's GPU.
Accuracy	Detection and ANPR figures come from small internal benchmarks, not certified evaluation. ANPR on real CCTV is harder than on the benchmark crops.
VMS coverage	ONVIF and vendor VMS adapters are not implemented beyond the interface and a mock.

20. Scalability Roadmap

FUTURE None of the items below is implemented. They are the intended path from a single-site deployment to a larger fleet.

1. **PostgreSQL as the default database** for any multi-camera deployment (the Compose path exists; make it the standard and load-test it).
2. **Separate AI worker processes / nodes**, each owning a set of cameras, coordinated through the database and Valkey.
3. **Event streaming** between ingestion, analytics and alerting, with a message bus, once one node is no longer enough.
4. **Edge AI** at camera clusters, sending events and clips instead of full video.
5. **More VMS adapters** (ONVIF, vendor SDKs) behind the existing adapter interface.
6. **Container orchestration** for the multi-node deployment.

21. Technology Stack

Layer	Technology
Frontend	Next.js 16, React 18, TypeScript, Tailwind CSS, Leaflet 1.9 with OpenStreetMap
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic settings, SQLAlchemy 2, Alembic
AI / video	Ultralytics YOLO11s, ByteTrack, YOLO plate detector, EasyOCR, OpenCV with FFmpeg, PyTorch (CUDA when available)
Data	SQLite (WAL); PostgreSQL 16 + PostGIS (Compose); Valkey (optional)
Security	bcrypt, JWT (python-jose), signed resource tokens, hash-chained audit
Operations	Docker, Docker Compose, Prometheus metrics, GitHub Actions (backend, frontend, Docker, CodeQL)

22. Testing & Verification

Check	Result
Backend test suite (pytest)	1101 passed, 18 skipped (after the contention fix, commit 795ebc8); 1096 passed after the map change (commit 607fc72)
Frontend unit tests (Vitest)	23 passed; TypeScript type check clean
Browser end-to-end tests (Playwright)	10 passed (login, live control room, plate search, vehicle, operator screens, camera control confirmation, console hygiene at phone and laptop widths)
Full-site click-through	34 pages and every visible button exercised on a copy of the database; no JavaScript errors, no page crashes, no server errors after fixes
CI (GitHub Actions)	Backend, Frontend, Docker (Compose with PostgreSQL + migration check) and CodeQL passed on PR #2
Regression tests for contention	Commit starved of a thread, event-loop freeze during an alert flush, snapshot inside a transaction: each fails on the old code and passes now

23. Conclusion

SENTINEL VISION already covers the full path a control room needs: cameras from mixed sources in one registry, AI detection, tracking and number-plate reading, rule-based alerts that grow into correlated incidents, evidence that can be verified and packaged, a camera map, and self-healing connectivity, all behind role-based access and an audit trail that shows tampering. The known limits are stated plainly: partial grid streaming, SQLite as a single-writer demo database, and a partly mapped camera fleet. The roadmap addresses them in order, starting with PostgreSQL as the standard database and separate AI workers.